

MLLM-Based UI2Code Automation Guided by UI Layout Information

FAN WU, Harbin Institute of Technology, Shenzhen, China

CUIYUN GAO*, Harbin Institute of Technology, Shenzhen, China

SHUQING LI, Chinese University of Hong Kong, China

XIN-CHENG WEN, Harbin Institute of Technology, Shenzhen, China

QING LIAO, Harbin Institute of Technology, Shenzhen, China

Converting user interfaces into code (UI2Code) is a crucial step in website development, which is time-consuming and labor-intensive. The automation of UI2Code is essential to streamline this task, beneficial for improving the development efficiency. There exist deep learning-based methods for the task; however, they heavily rely on a large amount of labeled training data and struggle with generalizing to real-world, unseen web page designs. The advent of Multimodal Large Language Models (MLLMs) presents potential for alleviating the issue, but they are difficult to comprehend the complex layouts in UIs and generate the accurate code with layout preserved. To address these issues, we propose LayoutCoder, a novel MLLM-based framework generating UI code from real-world webpage images, which includes three key modules: (1) Element Relation Construction, which aims at capturing UI layout by identifying and grouping components with similar structures; (2) UI Layout Parsing, which aims at generating UI layout trees for guiding the subsequent code generation process; and (3) Layout-Guided Code Fusion, which aims at producing the accurate code with layout preserved. For evaluation, we build a new benchmark dataset which involves 350 real-world websites named Snap2Code, divided into seen and unseen parts for mitigating the data leakage issue, besides the popular dataset Design2Code. Extensive evaluation shows the superior performance of LayoutCoder over the state-of-the-art approaches. Compared with the best-performing baseline, LayoutCoder improves 10.14% in the BLEU score and 3.95% in the CLIP score on average across all datasets.

CCS Concepts: • **Software and its engineering** → **Automatic programming**; • **Human-centered computing** → **User interface programming**.

Additional Key Words and Phrases: Web App Development, Automated Software Engineering, Multimodal Large Language Models

ACM Reference Format:

Fan Wu, Cuiyun Gao, Shuqing Li, Xin-Cheng Wen, and Qing Liao. 2025. MLLM-Based UI2Code Automation Guided by UI Layout Information. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA050 (July 2025), 23 pages. <https://doi.org/10.1145/3728925>

*Corresponding author.

Authors' Contact Information: Fan Wu, Harbin Institute of Technology, Shenzhen, Shenzhen, China, codenobuge@163.com; Cuiyun Gao, Harbin Institute of Technology, Shenzhen, Shenzhen, China, gaocuiyun@hit.edu.cn; Shuqing Li, Chinese University of Hong Kong, Hong Kong, China, sqli21@cse.cuhk.edu.hk; Xin-Cheng Wen, Harbin Institute of Technology, Shenzhen, Shenzhen, China, xiamenwxc@foxmail.com; Qing Liao, Harbin Institute of Technology, Shenzhen, Shenzhen, China, liaoqing@hit.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 2994-970X/2025/7-ARTISSTA050

<https://doi.org/10.1145/3728925>

1 Introduction



Fig. 1. Limitations of MLLMs.

Websites play a pivotal role in human digital life, serving as a channel for information retrieval and a platform for business transactions, social entertainment, and education. As of 2024, there are approximately 1.1 billion active websites worldwide, with around 252,000 new websites emerging daily [31]. Converting UI designs into code is a crucial step in web development, known as UI2Code. However, manually translating UI design into code is time-consuming and labor-intensive, requiring expert domain knowledge. The automation of UI2Code is essential to streamline this task, beneficial for improving the development efficiency.

The existing methods for UI2Code automation can be categorized into two types: one relies on traditional image processing techniques and predefined rules [24], and the other utilizes deep learning techniques [3, 7–10, 23, 43, 45]. Traditional image processing and rule-based methods involve manually crafting a set of rules or heuristics to detect and translate visual elements into code. For example, REMAUI [24] relies on a combination of edge detection, OCR (Optical Character Recognition) and predefined rules which is less adaptable to new design trends and layouts that are not explicitly defined in its rule set. On the other hand, the deep learning-based methods leverage deep learning techniques to train models capable of recognizing visual patterns and translating them into code, offering the potential to generalize across a wide range of UI designs. However, deep learning-based methods rely heavily on a large amount of curated and labeled training data and struggle with generalizing to real-world, unseen web page designs. For example, pix2code [7] integrates CNN and LSTM models to generate code from images. However, this method is effective only for simple user interfaces and can cover only five component types.

The advent of MLLMs [1, 5, 20, 22, 33] has introduced new possibilities for UI2Code automation while alleviating the reliance on large training data. However, MLLM-based UI2Code automation faces two key challenges: (1) **Hard to comprehend complex layouts.** As shown in Figure 1a, when faced with complex pages, GPT-4o refuses to restore the image, providing only simplified code. This indicates that GPT-4o has limitations in complex layout understanding. (2) **Hard to generate the accurate code with layout preserved.** As shown in Figure 1c, when provided with a webpage image, GPT-4o can generate the webpage code. However, there is obvious difference in the layout between the generated webpage and the reference webpage, suggesting that GPT-4o also has limitations in code generation for complex layouts.

To address these two challenges, we propose a novel MLLM-based framework aiming at generating code for real-world webpage images. The framework is composed of three modules: (1) Element Relation Construction, which aims at capturing UI layout by identifying and grouping components with similar structures; (2) UI Layout Parsing, which aims at generating UI layout

trees for guiding the subsequent code generation process; and (3) Layout-Guided Code Fusion, which aims at producing the accurate code with layout preserved.

We design a dataset called *Snap2Code*, which consists of two parts: *Seen* and *Unseen*. The *Seen* part is sourced from the globally top 500 most visited websites¹, while the *Unseen* part is from newly registered web domains of Shreshtait². Additionally, we select the open-source, high-quality dataset Design2Code [30]. We randomly sample 250, 250, and 100 web pages from these datasets for evaluation. Compared with the best-performing baseline, LayoutCoder improves 10.14% in the BLEU score and 3.95% in the CLIP score on average across all datasets between the reference and generated web pages. We further demonstrate that LayoutCoder maintains robust performance across web pages with varying layout complexity, and human evaluation results also validate the advantages of our approach.

In conclusion, our paper makes the following three contributions:

- We propose LayoutCoder, a novel MLLM-based framework for generating UI code from real-world webpage images.
- We design three modules in LayoutCoder: (1) Element Relation Construction (2) UI Layout Parsing and (3) Layout-Guided Code Fusion, which can improve the ability to understand complex layouts and generate the accurate code with layout preserved.
- Experimental Results demonstrate the superiority of LayoutCoder over the best baseline in both visual and code similarity on real-world web pages and robust generalization on highly complex web pages.

2 Related Work

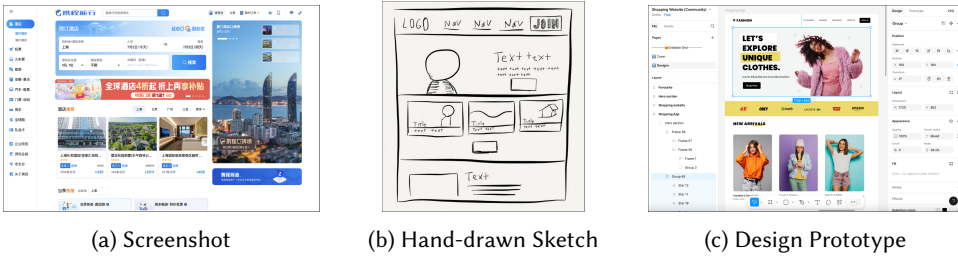


Fig. 2. Examples illustrating the difference between screenshots(a), hand-drawn sketches(b) and design prototypes(c).

2.1 UI2Code Automation

In the UI2Code task, based on the type of input, it can be divided into three categories: Screenshot2Code [7, 23, 45], which takes screenshots as input; Sketch2Code [6, 14, 28], which uses hand-drawn sketches; and Prototype2Code [4, 36], which uses visual design drafts. Among these, Screenshot2Code is the most challenging task. Compared to Prototype2Code, Screenshot2Code lacks metadata such as component hierarchies and attributes. Compared to Sketch2Code, it involves a larger number of elements, more complex layouts, and more diverse styles, as shown in Figure 2.

From an implementation perspective, UI2Code research can be categorized into three phases. **In the early phase**, traditional image processing techniques and heuristic rules were employed, with REMAUI [24] being a representative work. **In the middle phase**, research integrated computer vision (CV), natural language processing (NLP), and traditional machine learning algorithms, which can be divided into two types: the first type employed pure deep learning methods, such as pix2code [7], utilizing visual encoders like CNNs and text decoders like LSTMs and RNNs. The second type

¹<https://moz.com/top500>

²<https://shreshtait.com/newly-registered-domains/nrd-1m>

introduced hybrid methods that incorporated traditional machine learning algorithms, such as ReDraw [23], which used CNN and KNN, and FaceOff [45], which combined clustering and template-based similarity search. **In the later phase**, the focus shifted to large models (MLLMs), primarily divided into three categories: the first category is MLLM-based methods, such as DCGen [32], which adopts a multi-agent divide-and-conquer approach, and DeclarUI [47], which integrates DINO [41], SAM [17], and MLLMs. The second category involves fine-tuning MLLMs on UI datasets, such as WebSight VLM-8B [18], fine-tuned on synthetic datasets, and Design2Code-18B [30], fine-tuned on a mixture of synthetic and real data. The third category includes mainstream closed-source commercial MLLMs, such as GPT-4 [25] and Claude-3.5 [2].

Among the aforementioned works, only pix2code [7], DCGen [32], MLLMs fine-tuned on UI datasets [18, 30], and certain closed-source MLLMs focus on UI2Code automation for web applications, whereas the rest emphasize native mobile application development. Additionally, pix2code and WebSight VLM-8B primarily rely on synthetic datasets for training, while Design2Code-18B is trained on real-world web page data, and DCGen processes real-world data as input.

In our work, we aim to develop a novel framework based on MLLMs that indirectly enhances UI2Code performance by improving layout comprehension and code generation for complex layouts, specifically addressing the complex and diverse structures in real-world web pages.

2.2 UI Layout Generation

In the field of software engineering, work related to UI layouts is manifested through **UI Element Grouping**. This process builds upon UI Element Detection [38], grouping spatially close and functionally related elements. It effectively transforms fine-grained element-level blocks into coarse-grained group-level regions, reflecting the essence of layout. UI element grouping methods can be categorized into three main types: heuristic rules, clustering, and deep learning techniques. Screen Recognition [44] employs heuristic rules based on the alignment and distance characteristics of elements for grouping. Inspired by Gestalt design psychology [29], some researchers utilize spatial clustering and pairing for effective grouping. UIISCGD [37] leverages transformer to achieve grouping based on similar semantics. Screen Parsing [35] is inspired by Scene Graph Generation [40] in computer vision and constructs semantic relationships among elements using graph structures.

In the field of computer vision, research related to UI layouts manifests as **UI Layout Generation** which shares similarities with UI element grouping, as both tasks involve organizing and grouping UI components. Current layout generation techniques include methods based on GANs [21], VAEs [16], Transformers [15], and diffusion models [42] with studies conducted on mobile datasets such as RICO [11] and document layout datasets like PubLayNet [46]. Google's pix2struct [19] also explores extracting XML-like layout information from UI images. Unlike pix2struct, which is platform-agnostic, most of the aforementioned studies focus on layout extraction for mobile and document pages.

In response to the unique characteristics of web pages, we propose a layout generation method for web pages.

3 Methodology

3.1 Problem Formulation

This section provides a detailed definition of the UI2Code task that LayoutCoder aims to address. Let C_o denotes the original HTML + CSS code of a webpage, I_o denotes the screenshot of the webpage, and M be an MLLM. The UI2Code task takes the image I_o as input and outputs the generated HTML + CSS code $C_g = M(I_o)$, which then renders a screenshot I_g . The quality of the generated code C_g is

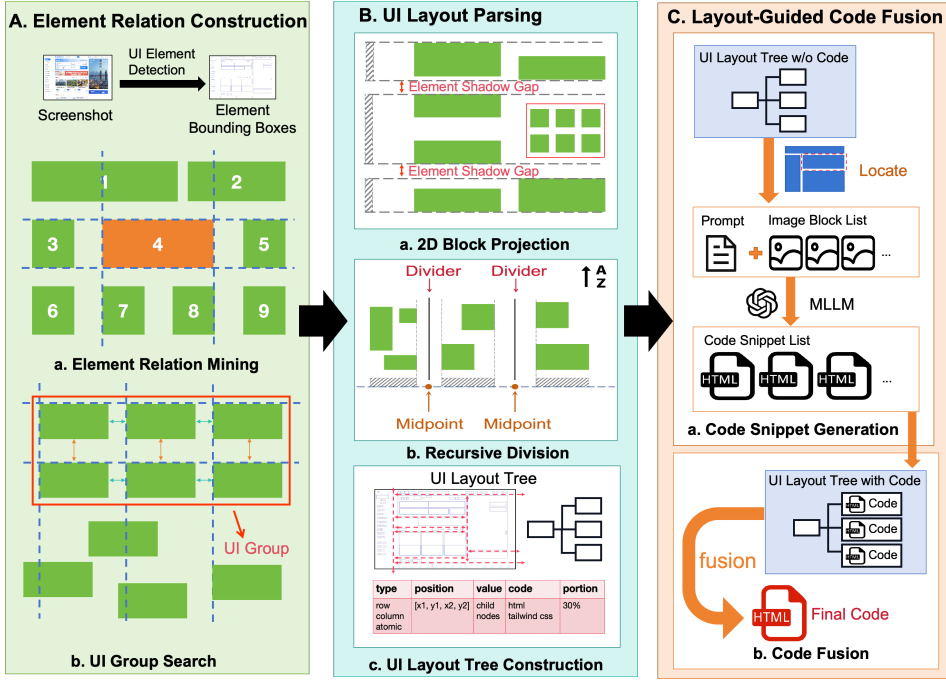


Fig. 3. The architecture of LayoutCoder.

evaluated based on both textual and visual similarity. Specifically, C_g should resemble C_o in terms of HTML's nested structure and tags, while I_g should visually align with I_o .

3.2 Overview

The proposed LayoutCoder framework consists of three major modules: Element Relation Construction, UI Layout Parsing, and Layout-Guided Code Fusion. Figure 3 illustrates the architecture of LayoutCoder. First, **Element Relation Construction** builds an element-relation graph based on UI element detection and searches for element groups that meet specific criteria. Next, **UI Layout Parsing** performs a recursive division of the webpage screenshot by identifying hidden dividing lines through block projection, generating a layout division diagram and parsing the screenshot into a UI layout tree. Finally, during the **Layout-Guided Code Fusion** phase, the screenshot is divided into atomic image blocks based on location information from UI layout tree, and code snippets are sequentially generated for each image block with the help of Multimodal Large Language Models (MLLMs). These code snippets are then fused into the final synthesized code with the guidance of the UI layout tree.

3.3 Module A: Element Relation Constuction

The goal of this module is to address the challenge of grouping UI elements that exhibit similar structures across a webpage. Through empirical analysis, we have observed that modern web designs frequently incorporate repetitive regions with analogous structures. If these repetitive regions are not grouped and merged, directly participating in segmentation in UI Layout Parsing could lead to four potential issues: 1) excessive splitting of the layout, 2) generation of numerous redundant image blocks, 3) inconsistent code generated from redundant blocks, and 4) increased API calls per task. Hence, we propose two methods, Element Relation Mining and UI Group Search,

to solve the problem of grouping repetitive regions with similar structures from the perspective of element spatial relationships.

Algorithm 1: UI Group Search

Input :List of boxes *boxes*
Output:List of groups *groups*
Function SearchUIGroups(*boxes*):

```

    visited  $\leftarrow \emptyset$ ;
    groups  $\leftarrow \emptyset$ ;
    while  $|visited| < |boxes|$  do
        start_bbox  $\leftarrow$  ChooseRandomBBox(boxes);
        visited  $\leftarrow$  Append(visited, start_bbox);
        // (1) Expand group via BFS for valid nodes.
        group  $\leftarrow$  ExpandGroup(start_bbox, boxes, visited);
        groups  $\leftarrow$  Append(groups, group);
    end
    return groups;

Function ExpandGroup(start_bbox, boxes, visited):
    group  $\leftarrow$  start_bbox;
    queue  $\leftarrow$  start_bbox;
    while queue  $\neq \emptyset$  do
        current_bbox  $\leftarrow$  Pop(queue);
        // (2) Get neighbors from pre-built relation graph (Sec 3.3.1)
        neighbors  $\leftarrow$  GetBBoxNeighbors(current_bbox);
        // (3) Traverse neighbors by alignment/spacing rules.
        for neighbor in neighbors do
            if neighbor  $\notin$  visited and CanAddToGroup(neighbor, group) then
                group  $\leftarrow$  Append(group, neighbor);
                queue  $\leftarrow$  Append(queue, neighbor);
                visited  $\leftarrow$  Append(visited, neighbor);
            end
        end
    end
    return group;

```

3.3.1 Element Relation Mining. LayoutCoder first applies the UI element detection algorithm (UIED) [38] to extract the bounding boxes of all text and non-text elements from the webpage image. Additionally, to ensure that the obtained bounding boxes do not overlap, we sort the bounding boxes by *row_min* and *column_min*, and calculate four values—*inter_area*, *iou*, *ioa*, and *iob*—between adjacent bounding boxes. Based on these values, we determine the type of overlap: if it's an inclusion relationship, the largest bounding box is retained; if it's an intersection relationship, the smallest enclosing rectangle is calculated. Once the bounding boxes are obtained, we observe that each element has varying numbers of neighboring elements in different directions (i.e., above, below, left, and right). For example, in Figure 3 A-a, element 4's direct upper neighbors are elements 1 and 2, while its direct left neighbor is element 3. These neighboring relationships can be represented as edges in a graph, where UI elements function as nodes. To mine these spatial relationships, we traverse all the bounding boxes and construct a graph that encodes the connections between each element and its direct neighbors. This relationship graph forms the foundation for subsequent layout parsing and ensures that structurally similar regions are appropriately grouped, thereby reducing redundancy and improving the consistency of the generated code.

3.3.2 UI Group Search. With the relationship graph generated from Element Relation Mining, we proceed to identify valid groupings. Prior studies indicate that UI elements with similar structures

commonly exhibit alignment and consistent spacing patterns [39, 44]. Research in UI grouping has also explored spatial clustering methods, such as DBSCAN [12], and principles from Gestalt psychology for effective pairing and grouping [29]. For our grouping process, alignment and spacing criteria are defined as follows: 1) Alignment: Both text and non-text elements are aligned to the left, ensuring a consistent left-to-right flow within the layout; 2) Spacing: The vertical and horizontal spacing between elements is consistent. However, for groups with fewer than or equal to two elements in either the vertical or horizontal direction, spacing is not a strict requirement. Instead of using complex semantic grouping methods, we employ a simpler heuristic based on alignment and spacing. All elements are considered potential grouping candidates. Starting with a randomly selected element, we traverse its neighboring elements in the relationship graph, identifying those that meet alignment and spacing criteria. Using a breadth-first search approach, we iteratively expand the current group by adding eligible neighboring elements until the queue is empty. This process repeats for all unvisited elements until every element has been processed. Algorithm 1 describes this grouping process, while Figure 3 A-b illustrates an example where six elements are grouped based on alignment and equal spacing conditions, forming a structured grid layout.

3.4 Module B: UI Layout Parsing



Fig. 4. Illustration of segmentation by SAM(b) and LayoutCoder(c).

This module addresses the challenge of accurately segmenting the layout of UI images. Before designing this module, we conducted some preliminary studies on layout segmentation using existing image segmentation techniques, such as SAM. Here we conduct a case study based on taobao.com, highlighting the limitations of SAM in UI images. As illustrated in Figure 4(b), our analysis identified several challenges: 1) Difficulty in preserving small UI components, referring to the green box in Figure 4(b); 2) Object sensitivity in UI images, referring to the red box in Figure 4(b); 3) Failure to cluster repeated UI components of identical types, referring to the black box in Figure 4(b). To alleviate the above limitations, we propose three methods within the LayoutCoder framework, (1) 2D Block Projection, (2) Recursive Division, and (3) UI Layout Tree Construction, to segment UI images and capture layout information generated during the segmentation process.

3.4.1 2D Block Projection. As shown in Figure 3 B-a, projecting element blocks generates projection intervals that correspond to the positions of dividing lines. The projection process can merge overlapping elements in the same direction, facilitating the identification of blank regions and preventing element truncation during segmentation. For instance, as illustrated in Figure 5b (a *ctrip.com* homepage example), the bounding boxes extracted by UIED are represented in rectangle

boxes, while the blue and red areas visualize the block projection process, indicating potential layout divider lines' positions. After grouping elements in Element Relation Constuction, we project the blocks horizontally and vertically, collecting the projection intervals as segmentation targets for Recursive Division.



Fig. 5. An example of UIED and 2D Block Projection processing a webpage image.

3.4.2 Recursive Division. Using the horizontal and vertical intervals collected in 2D Block Projection, we sort them in descending order of distance. The rationale for this approach lies in the proximity principle of Gestalt design psychology [29]: elements that are closer together are perceived as related, while those further apart are seen as separated. After sorting the intervals, we first select the projection region with the largest interval and perform a midpoint division, following the proximity principle. This recursive process splits the layout into multiple regions. Each region is treated as an initial block and subjected to further projection, sorting, and division until no further divisions are possible. Figure 3 B-b shows an example where the midpoint of the interval is selected as the dividing point, and the corresponding line becomes the target division line. The effectiveness of this method can be referenced in Figure 4c.

3.4.3 UI Layout Tree Construction. We observed that the recursive segmentation process corresponds to a tree structure, where the resulting segments are nodes of the tree. This tree structure shares similarities with the HTML DOM tree. Therefore, during the segmentation process in Recursive Division, we store the layout tree T_{layout} . This tree facilitates accurate restoration of the original layout and supports the subsequent code generation and fusion process. As shown in Figure 3 B-c, we designed an attribute table that records each node's type, position, child nodes, code block, and spatial ratio. Nodes are classified into two main types: container nodes (row and column containers representing layout relationships between regions) and atomic nodes (indivisible regions). The position attribute records each region's bounding box, and the code block attribute reserves space for storing code snippets for each region.

3.5 Module C: Layout-Guided Code Fusion

This module focuses on code generation and fusion. We observe that DCGen uses MLLMs for code fusion, but this approach is limited by (1) the ability of text to express layout is constrained, and (2) MLLMs have limitations in code generation for complex layouts. To address this, we divide the overall code generation task into two sub-tasks. The first sub-task deals with code generation for atomic regions within the leaf nodes of UI layout tree, and the second sub-task focuses on handling the layout relationship between code snippets within leaf nodes of UI layout tree. For each indivisible atomic region (i.e., each leaf node of UI layout tree), we use MLLMs to generate code snippets. After finishing the code snippet generation of each leaf node, we convert the UI

layout tree without code into the UI layout tree with code attached to the leaf nodes. Finally, by the guidance of the layout relationship within the UI layout tree, code snippets within leaf nodes are fused into the final code.

This approach leverages the strengths of MLLMs in generating UI code for simple web pages while mitigating their limitations in code generation for web pages with complex layouts.

Algorithm 2: Code Fusion

Input : JSON layout structure *structure*
Output: HTML and CSS representation *html*
Function CodeFusion(*root_node*):
 // (1) Traverse from the root node.
 return ProcessNode(*root_node*, True);
Function ProcessNode(*node*, *is_root*):
 Initialize string *html* as empty;
 if *node*["type"] = 'row' **then**
 // (2) Single Div Generation.
 html ← Append(*html*, "<div class='row'>");
 html ← Append(*html*, " root" if *is_root* else "");
 html ← Append(*html*, " style='flex: ", *node.get*("portion", 1), "; ">\n");
 // (3) Multi-Layer Div Generation
 foreach *child* in *node.value* **do**
 html ← Append(*html*, ProcessNode(*child*));
 end
 html ← Append(*html*, "</div>\n");
 else if *node*["type"] = 'column' **then**
 // (2) Single Div Generation.
 html ← Append(*html*, "<div class='column'>");
 html ← Append(*html*, " root" if *is_root* else "");
 html ← Append(*html*, " style='flex: ", *node.get*("portion", 1), "; ">\n");
 // (3) Multi-Layer Div Generation
 foreach *child* in *node.value* **do**
 html ← Append(*html*, ProcessNode(*child*));
 end
 html ← Append(*html*, "</div>\n");
 // (4) Connect the div element for the leaf node of code snippet
 else if *node*["type"] = 'atomic' **then**
 html ← Append(*html*, "<div class='atomic' style='flex: ", *node*["portion"], "; ">, *node*["code"],
 "</div>\n");
 end
 return *html*;

3.5.1 Code Snippet Generation. This method handles the first sub-task: code generation for indivisible atomic regions. In the UI Layout Parsing Module, we mentioned that the layout tree's attribute table includes a "position" attribute that stores each atomic region's bounding box. A webpage image contains multiple atomic regions, and by locating their bounding boxes, we can crop out the image blocks to be processed. The prompt for code snippet generation, combined with the image blocks, is batched into the MLLM, generating code snippets C_{part}^i for each block. Figure 3 C-a illustrates the process. The prompt for code snippet generation differs from that for complete HTML code generation. Specifically, we impose the following core requirements when designing the prompt for MLLM: 1) The generated code must start with a <div> tag and end with a </div> tag; 2) The code should not impose fixed width and height settings; 3) The image proportions should be preserved; 4) The margin and padding should be set to 0; 5) Make sure the generated code doesn't conflict with outer divs in layout and style. The first requirement ensures that the code

generated represents a code snippet rather than a complete HTML code. The second and third requirement avoids MLLM's tendency to set fixed sizes and ignore the proportion when restoring images. The fourth and fifth ensure smooth integration into the layout code, which forms the basis of div decoupling that ensures no conflict between inner and outer divs.

After finishing the code snippet generation of all atomic regions, code snippets are attached to leaf nodes of the UI layout tree. The initial UI layout tree without code is transformed into a UI layout tree with code as shown in [Figure 3 C-b](#).

3.5.2 Code Fusion. This method handles the second sub-task: leverage the layout relationship between leaf nodes of the UI layout tree to fuse code snippets. In [Algorithm 2](#), **(1) Traversal:** We first traverse the nodes of the UI Layout tree layer by layer, handling non-leaf and leaf nodes separately. Non-leaf nodes are container nodes of type row or column, while leaf nodes are atomic nodes of type atomic. **(2) Single Div Generation:** Then, each node in the UI Layout Tree eventually generates a corresponding div. Since there is a decoupling relationship between inner and outer div elements in HTML, there is also a decoupling relationship between nodes. Based on this property, we control the spatial arrangement of nodes within different container nodes through associated styles using class, and control the spatial proportion of the corresponding div through inline styles. **(3) Multi-Layer Div Generation:** Layer by layer, the nodes of the UI Layout Tree are converted into multi-layer nested divs that can represent the UI Layout, with the leaf nodes containing local code snippets placed under the corresponding 'div' of their container nodes. When the depth-first traversal is complete, the UI Layout Tree with code snippets is transformed into the final code.

4 Experimental Setup

4.1 Research Questions

In this section, we evaluate the effectiveness of LayoutCoder by comparing it with the state-of-the-art approaches. We mainly focus on the following Research Questions (RQs):

RQ1: How effective is LayoutCoder compared with existing UI2Code methods?

RQ2: What is the impact of key components on the performance of LayoutCoder for UI2Code?

RQ3: What is the influence of the complexity of real-world websites on the performance of LayoutCoder?

RQ4: What are the results of user studies and human evaluations?

To answer **RQ1**, we compare the five representative models for UI2Code, including pix2code, WebSight VLM-8B, DeepSeek VL-7B, GPT-4o and Claude 3.5 Sonnet. To answer **RQ2**, we explore the impact of three key components of LayoutCoder to conduct ablation studies, including UI Grouping, Gap Sorting, Custom Prompt of code snippet generation(Prompt). This allows us to evaluate the independent contributions of each component to the overall framework. To answer **RQ3**, we select four representative metrics to measure website complexity: the depth of the DOM tree, the number of DOM tags, the number of tokens in the HTML code, and the aspect ratio of images. We evaluate the impact of increasing web page complexity on LayoutCoder. To answer **RQ4**, we conduct a human evaluation to qualitatively evaluate the web pages generated by the best-performing baseline Claude 3.5 Sonnet(Self-Refine) and LayoutCoder. In total, three participants with experience in software development or UI design are invited to take part in the evaluation.

4.2 Datasets

In this work, we evaluate LayoutCoder on two real-world UI-to-Code datasets: Design2Code [30] and Snap2Code. Specifically, **(1) Design2Code:** Design2Code [30] is an open-source, high-quality dataset composed of 484 real-world web pages from the C4 validation set, primarily used to test MLLMs in converting visual designs to code. The collection of 250 website samples from

Table 1. The comparison of Snap2Code with Design2Code: scope, complexity, and applicability.

	Scope	Sample Size	Complexity	Applicability
Design2Code	484 real-world web pages from the C4 validation set	250	Simple(All images replaced with placeholders)	Web UI-to-code
Snap2Code(Seen)	the top 500 globally visited websites	250	Complex(Original images retained)	Web UI-to-code
Snap2Code(Unseen)	newly registered web domains from the past month as of January 3, 2025	100	Moderate(Original images retained)	Web UI-to-code

Design2Code dataset contains an average of 158 tags, where each HTML code file has an average length of 625 tokens and each web page image has an average aspect ratio of 1.1338, as shown in Table 2. It is important to note that in the Design2Code dataset, all images on the web pages have been replaced with placeholder images. **(2)Snap2Code:** To evaluate the performance on web pages with more complex layouts and components, we also created a new dataset called Snap2Code. Furthermore, to comprehensively assess the performance of our framework on unseen datasets, the Snap2Code dataset is divided into two parts: *Seen* and *Unseen*. The *Seen* portion is compiled from the top 500 globally visited websites, as ranked by Moz³, while the *Unseen* portion consists of 100 newly registered web domains from the past month (as of January 3, 2025), sourced via Shreshtait⁴. We use the Selenium framework⁵ and GeckoDriver⁶ to ensure high fidelity and accuracy in capturing webpage data. During the collection process, any web pages that were blocked, denied access, blank, or failed to load were excluded to maintain the quality of the dataset. For the *Unseen* portion of the dataset, we checked the first archive time of the collected webpages using the API of The Wayback Machine⁷ to ensure that all webpages were created after August 7, 2024, making them unseen by the selected version of MLLM (e.g., GPT-4o-2024-08-06, Claude-3-5-Sonnet-202406).

Comparison of Dataset Complexity. Table 1 illustrates the differences between the Design2Code and Snap2Code datasets in terms of scope, complexity, and applicability. Table 2 mainly shows the complexity of the image and text data in Design2Code and Snap2Code from a data statistics perspective. Upon observation, we find that Snap2Code(Seen) exceeds Design2Code by an order of magnitude in the median values for DOM Breadth, Tag Count, and Length. In addition, the median value for the Aspect Ratio metric of images in Snap2Code (Seen) is twice that of Design2Code. This metric poses a challenge for methods aimed at understanding longer images and generating longer code. Additionally, Snap2Code retains the original images within the web pages, presenting a more realistic and challenging scenario for UI2Code task, as shown in Figure 6.

Clarification of Snap2Code(Seen) Dataset Size Inconsistency. Although Snap2Code(Seen) is compiled from the top 500 globally visited websites, some samples were excluded due to issues encountered during web crawling, such as being blocked, denied access, displaying blank pages, or failing to load. Additionally, considering that the GPT-4o and Claude 3.5 Sonnet models are quite expensive, we set the dataset size to 250, following the methods from previous works [24, 32, 47], which led to this Snap2Code(Seen) dataset size inconsistency.

4.3 Baselines

To evaluate the performance of LayoutCoder, we select five representative models (pix2code [7], WebSight VLM-8B [18], DeepSeek VL-7B, GPT-4o, and Claude 3.5 Sonnet) and three different levels of prompts (Direct, CoT, and Self-Refine) targeting closed-source models, resulting in a total of nine baselines. The **pix2code** integrates CNN and LSTM models to generate code from images, which is one of the few representative open-source models available for UI-to-Code tasks in web applications. **WebSight VLM-8B** is an open-source VLM (vision-language model) from Huggingface, fine-tuned on a synthetic UI-to-Code dataset, WebSight-v0.1⁸. **DeepSeek VL-7B** is an

³<https://moz.com/top500>

⁴<https://shreshtait.com/newly-registered-domains/nrd-1m>

⁵<https://www.selenium.dev/>

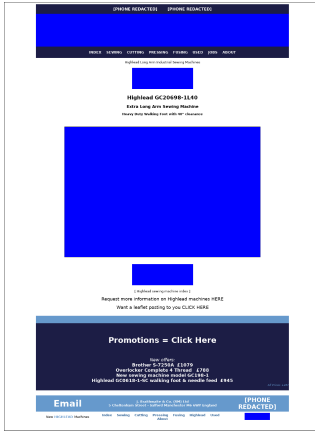
⁶<https://github.com/mozilla/geckodriver>

⁷<https://web.archive.org/>

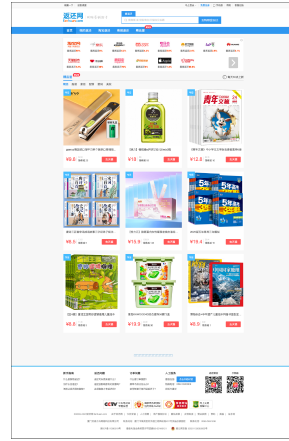
⁸<https://huggingface.co/datasets/HuggingFaceM4/WebSight>

Table 2. The statistics of both Design2Code and Snap2Code.

Dataset	Metric	DOM Depth	DOM Breadth	Tag Count	Unique Tags	Length	Aspect Ratio
Design2Code	Mean	13	80	158	23	625	1.13
	Std	5	51	100	6	338	0.50
	Min	5	6	13	10	34	0.56
	Median	12	67	134	23	571	1.04
	Max	26	260	486	46	1679	2.60
Snap2Code(Seen)	Mean	20	702	1468	37	4379	2.74
	Std	7	772	1580	11	5029	1.53
	Min	6	6	14	11	68	0.56
	Median	20	524	1118	37	3107	2.79
	Max	43	8787	15671	94	53682	5.98
Snap2Code(Unseen)	Mean	18	168	332	30	1077	1.42
	Std	7	164	326	11	1284	0.98
	Min	7	13	25	13	44	0.59
	Median	17	116	214	29	585	0.96
	Max	45	963	1888	60	6701	5.16



(a) Design2Code



(b) Snap2Code

Fig. 6. Two examples from Design2Code(a) and Snap2Code(b).

open-source Vision-Language model with broad multimodal understanding, capable of processing diverse real-world applications. **GPT-4o**, developed by OpenAI, is a closed-source multimodal large language model (MLLM) that excels in image-to-text and UI-to-Code tasks, which serves as a key component in the Code Fusion module of the LayoutCoder framework. Similarly, **Claude 3.5 Sonnet**, developed by Anthropic, offer exceptional capabilities in code generation and visual understanding.

For GPT-4o, we set the model to version "GPT-4o-2024-08-06," while for Claude 3.5 Sonnet, the version is "Claude-3-5-Sonnet-20240620" both with a maximum token limit of 4096 and a temperature of 0. Additionally, The three different levels of prompts are based on the prompt schemes from Design2Code and DCGen. We apply them to GPT-4o and Claude 3.5 Sonnet. The specific content of the prompts will be made available in the anonymous code repository. In this paper, we will refer to Claude(D), Claude(C), and Claude(SR) as Claude 3.5 Sonnet+Direct, Claude 3.5 Sonnet+CoT, and Claude 3.5 Sonnet+Self-Refine, respectively, with similar naming for GPT-4o.

4.4 Metrics

Following the previous work [30, 32, 47], we employ two widely-used metrics: BLEU score [26] and CLIP score [13], to evaluate the similarity between the generated webpage code and the rendered webpage focusing on textual and visual aspects.

To evaluate the textual similarity, we use the **BLEU score** to evaluate the similarity between the generated code (C_g) and the original code (C_o). The BLEU score is a widely used metric in natural language processing to assess the similarity of generated text to reference text based on n-gram matching. It is computed as: $BLEU - N = BP \times \exp \left(\sum_{n=1}^N w_n \log P_n \right)$ where P_n denotes the n-gram precision, and w_n represents the weight for each n-gram. N is the maximum n-gram order. Brevity Penalty (BP) is applied to penalize generated texts that are significantly shorter than the reference text. If the generated text length c is shorter than the reference text length r , $BP = \exp \left(1 - \frac{r}{c} \right)$; otherwise, $BP = 1$.

To evaluate the visual similarity, we use the **CLIP score** to assess the similarity between the webpage screenshot rendered from the generated code (I_g) and the screenshot rendered from the original code (I_o). CLIP score is a metric in computer vision that evaluates the visual similarity between generated and reference images. The images are converted into embedding vectors by CLIP's image encoder, and the cosine similarity between these vectors determines their similarity in the CLIP embedding space [27]. In particular, we extract features using the ViT-B/32 model.

4.5 Implementation Details

For the execution environment, we run Python programs with Python 3.8.18 and manage dependencies using conda 24.5.0. All experiments are conducted on a Linux server (64-bit Ubuntu 20.04) equipped with two 48-core Intel Xeon Platinum 8260C CPUs @ 2.40GHz (96 logical processors), four NVIDIA A100-40GB GPUs, and 251GB RAM.

5 Experimental Results

This section presents our experimental results and answers the four RQs outlined in Section 4.1.

5.1 RQ1: Comparison with Baselines

To answer this RQ, we compared the performance of LayoutCoder with the nine baselines. Table 3 shows the experimental results of each method of BLEU and CLIP scores. Additionally, Figure 7a and Figure 7b present box plots of the BLEU and CLIP scores, providing further insights into the distribution of results across the two datasets.

Analyses:

(1) **LayoutCoder outperforms other baselines in terms of both text similarity and visual fidelity.** When considering all the two metrics regarding the three datasets (6 combination cases altogether), LayoutCoder has the best performance in 5 out of the 6 cases. As shown in Table 3, it has a 24.12% improvement in BLEU score on the Snap2Code(Seen) dataset and a 2.28% improvement on the Design2Code dataset. For visual consistency, LayoutCoder outperforms Claude(SR) by 8.38% to 4.63% in terms of CLIP score across the Seen and Unseen parts of the Snap2Code dataset. This indicates that LayoutCoder achieves better performance in both text similarity and visual fidelity. On the unseen data, while LayoutCoder shows a slight degradation compared to the seen dataset, its overall performance remains optimal. Furthermore, the Claude model exhibits the best CLIP score on the Design2Code dataset. It may be due to its advantages in visual comprehension and code generation capabilities.

(2) **LayoutCoder also demonstrates advantages in the overall data distribution compared to other methods.** The Figure 7a illustrates that LayoutCoder's lower quartile is higher than Claude(SR)'s upper quartile in the BLEU score on the Snap2Code(Seen) dataset which indicates that over 75% of the web pages generated by LayoutCoder surpass 75% of those generated by Claude(SR). On the Design2Code dataset, 50% of the web pages generated by LayoutCoder also surpass the vast majority of Claude(SR)'s, excluding some outliers. Additionally, Figure 7b visualizes the CLIP score

Table 3. Comparison results of LayoutCoder with three baseline methods(%).

Method	Design2Code		Snap2Code(Seen)		Snap2Code(Unseen)	
	BLEU	CLIP	BLEU	CLIP	BLEU	CLIP
Claude(D)	2.19	81.90	1.29	70.77	2.11	71.35
Claude(C)	2.31	81.17	1.39	71.66	2.21	73.38
Claude(SR)	2.66	82.74	1.95	71.96	2.45	71.51
GPT-4o(D)	1.86	75.57	0.97	71.37	1.34	68.11
GPT-4o(C)	2.04	77.99	1.14	71.22	1.59	70.26
GPT-4o(SR)	1.96	76.70	1.00	71.19	1.39	70.56
pix2code(greedy)	1.59	68.24	1.16	68.36	2.47	64.79
WebSight VLM-8B	1.75	76.40	1.07	70.41	1.05	66.86
DeepSeek VL-7B	0.69	72.34	0.45	66.64	0.49	66.65
LayoutCoder	4.93	81.58	26.07	80.35	6.46	76.14

distribution, which shows 25% of the web pages generated by LayoutCoder exceeds 85.00% in the CLIP score on the Snap2Code(Seen) dataset, while only 25% of Claude(SR)'s approaches 80.00%.

Thus, LayoutCoder not only excels in average metrics but also in the overall data distribution compared to other methods.

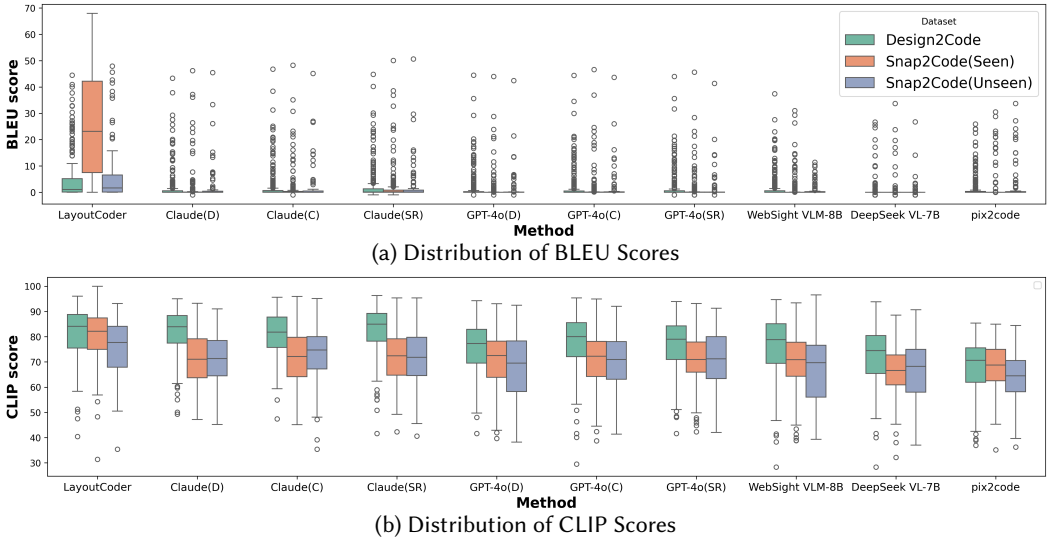


Fig. 7. Box plots showing the distribution of BLEU and CLIP scores for LayoutCoder and baseline methods.

Answer to RQ1: In comparison with the best baseline, Claude(SR), LayoutCoder achieves the best performance across all evaluated performance metrics, with the improvement of 10.14% in the BLEU score and 3.95% in the CLIP score on average across all datasets.

5.2 RQ2: Ablation Study

To answer RQ2, we explore the impact of different components of LayoutCoder, including the UI Group Search (i.e, w/o UI grouping), Recursive Division (i.e, w/o Gap Sorting), and Code Snippet Generation (i.e, w/o Prompt). Table 4 presents the performance of LayoutCoder and its three variants on Snap2Code and Design2Code datasets.

5.2.1 UI Group Search. To investigate the impact of the UI grouping in element relation construction, we deploy one variant that removed UI grouping (i.e., w/o UI Group Search). It means that similar structural regions are not merged in a UI group. As shown in Table 4, we can find that the UI grouping can improve the performance of LayoutCoder. Specifically, in the Snap2Code(Seen) dataset, the BLEU score increased by 0.55%, the CLIP score increased by 1.51%. When UI elements

Table 4. Performance of LayoutCoder and its variants on two datasets(%).

Dataset	Design2Code		Snap2Code(Seen)		Snap2Code(Unseen)	
Metrics	BLEU	CLIP	BLEU	CLIP	BLEU	CLIP
w/o Grouping	4.76	81.21	25.53	78.84	6.13	74.25
w/o Gap Sorting	4.63	80.06	25.89	74.05	6.19	71.47
w/o Prompt	4.51	78.25	26.04	75.51	6.44	70.05
LayoutCoder	4.93	81.58	26.07	80.35	6.46	76.14

are not grouped, structurally similar regions lead to inconsistencies in code generation. This lack of grouping disrupts both layout segmentation and visual rendering uniformity.

5.2.2 Recursive Division. To explore the impact of the recursive division in UI layout Parsing, we conduct a variant, which only uses the division without gap sorting (i.e., w/o Recursive Division), thereby disrupting the far-to-near segmentation order that adheres to the Gestalt principle of proximity. As shown in Table 4, we can find that the gap sorting can improve the performance of LayoutCoder. Specifically, in the Snap2Code(Seen) dataset, the BLEU score increased by 0.19% and the CLIP score increased by 6.30%. When the element shadow gaps are not sorted, unsorted gaps lead to unordered segmentation. This lack of gap sorting disrupts both layout segmentation and the accurate code generation.

5.2.3 Simplifying the Prompt. To explore the impact of div decoupling in the prompt of code snippet generation, we employ a variant that simplifies its basic form in layout-guided code fusion, thereby disrupting the compatibility of layout and style between outer divs and inner divs. As shown in Table 4, we can find a complete prompt can improve the performance of LayoutCoder. Specifically, in the Snap2Code(Seen) dataset, the BLEU score increased by 0.03% and the CLIP score increased by 4.83%. When the prompt is simplified, layout and style of inner divs conflicts with those of outer divs. This lack of the complete prompt disrupts code generation and visual rendering uniformity.

Answer to RQ2: All three modules are essential to the LayoutCoder framework, with gap sorting (i.e., Recursive Division) and the complete prompt (i.e., Code Snippet Generation) contributing the most to overall performance.

5.3 RQ3: Influence of Website Complexity

To answer RQ3, we conduct experiments of webpage complexity to evaluate the performance of LayoutCoder, against the leading baseline, Claude(SR). We extract webpage code and rendered image features that effectively encapsulate the complexity of a webpage. Specifically, we identified and selected four representative features, including DOM depth, tag count, length and aspect ratio.

DOM Depth: The depth of the Document Object Model (DOM) tree depth in the webpage's HTML files, representing the nesting complexity of HTML tags; **Tag Count:** We quantified the structural complexity of HTML documents by counting the number of tags within each file; **Length (tokens):** The number of tokens in the webpage's HTML code, measured using the cl100k_base tokenizer from OpenAI's tiktoken library; **Aspect Ratio:** The aspect ratio of the webpage images, with all images fixed at a width of 1512 pixels, reflects visual complexity.

Due to uneven sample densities across various complexity intervals, we implement equal-frequency binning on the x-axis to ensure a uniform distribution of data points across the range of complexities. Figure 8 illustrates the performance of LayoutCoder, compared to Claude(SR) across four distinct complexity features. Each subfigure within the figure provides a detailed visual representation of how LayoutCoder performs relative to Claude(SR) under each specific complexity feature, evaluating the effectiveness in handling varying levels of website complexity.

Analyses:

(1) **LayoutCoder consistently outperforms Claude(SR) across different webpage complexity dimensions in the Snap2Code dataset.** As shown in Figure 8, we can find that it is evident that LayoutCoder maintains a performance gap with Claude(SR) across varying levels of complexity in the Snap2Code dataset, showcasing LayoutCoder’s advantage on complex web pages. For example, LayoutCoder achieves an 12.50% improvement in the CLIP score on the Snap2Code(Seen) dataset for tag count in the range (2265, 2817.6] and a 15.00% improvement for aspect ratio in the range (4.213, 4.963]. In addition, it also achieves a 15.00% improvement in the BLEU score on the Snap2Code(Seen) dataset for length(tokens) in the range (3977.6, 5259.9].

(2) **LayoutCoder has robust generalization on highly complex web pages.** As the website complexity increases, LayoutCoder consistently maintains a high score in both BLEU and CLIP scores. For example, on the Snap2Code(Seen) dataset, the performance of LayoutCoder consistently surpasses 10.00% in the BLEU score and 74.00% in the CLIP score.

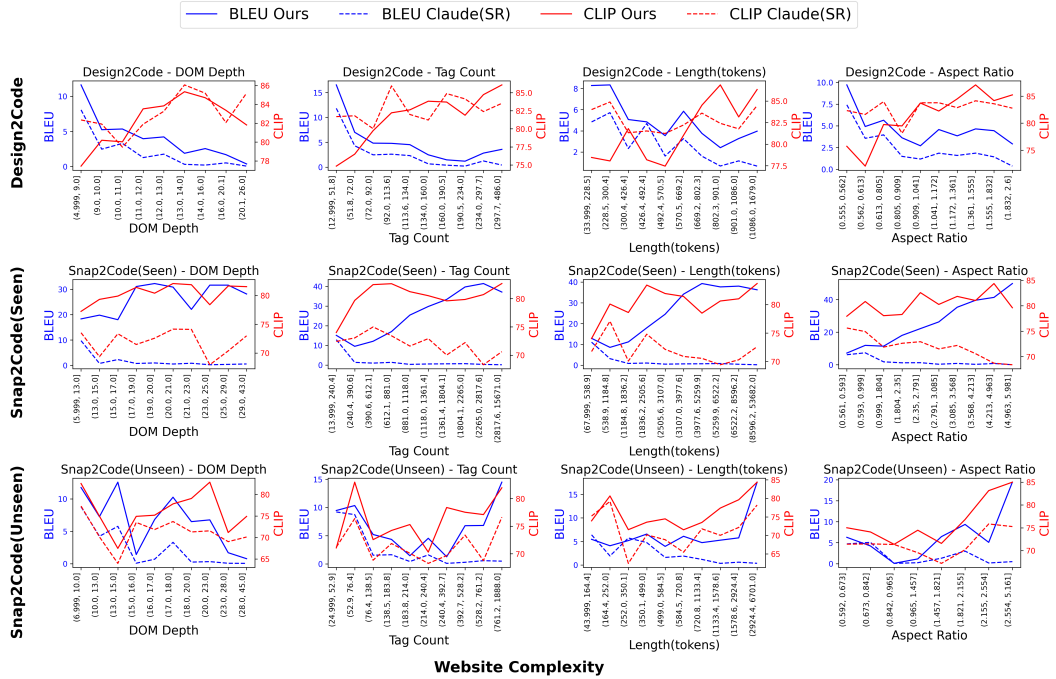


Fig. 8. The performance of LayoutCoder across the four complexity features for both datasets.

Answer to RQ3: LayoutCoder outperforms Claude(SR) across different levels of webpage complexity and demonstrates robust generalization and stability on highly complex web pages.

5.4 RQ4: Human Evaluation

In this section, we perform a quantitative evaluation of web pages generated by Claude 3.5 Sonnet(Self-Refine) and LayoutCoder through human evaluation. The evaluation is conducted through online questionnaire. Specifically, four participants with experience in software development or UI design are invited to take part in the evaluation. Each participant is asked to answer 120 questions, containing a total of 360 images, and rate the quality of web pages generated by Claude 3.5 Sonnet(Self-Refine) and LayoutCoder.

5.4.1 Survey Design. We randomly selected 50, 50 and 20 web pages from the Design2Code, Snap2Code(Seen) and Snap2Code(Unseen) datasets, respectively, for evaluation. Each question in

the survey comprises a reference webpage image along with webpage images generated by the two methods, as well as the corresponding webpage code generated by both methods. Each participant will be given 120 questions. To mitigate order bias, the display order of webpage images generated by different methods will be randomized within each question. The quality of the generated web pages is evaluated based on four metrics: Layout Accuracy, Visual Fidelity, Content Alignment and Code Usability, using a 1-to-5 Likert scale (5 for excellent, 4 for good, 3 for acceptable, 2 for marginal, 1 for poor). At the beginning of the survey, we explained the meaning of the two evaluation metrics: Layout Accuracy measures how closely the layout structure of the generated page and the reference page; Visual Fidelity evaluates the overall visual consistency of the generated page and the reference page; Content Alignment measures the similarity between the specific content (e.g., text, images) in the generated page and the reference page; Code Usability evaluates how helpful the generated code is for developers.

Table 5. Human evaluation on web pages generated by LayoutCoder and Claude(SR).

Dataset	Metrics	Claude(SR)	LayoutCoder
Design2Code	Layout Accuracy	3.25	3.77
	Visual Fidelity	3.22	3.77
	Content Alignment	3.39	4.01
	Code Usability	3.51	4.01
Snap2Code(Seen)	Layout Accuracy	1.64	4.07
	Visual Fidelity	1.64	4.13
	Content Alignment	1.86	4.12
	Code Usability	1.75	4.23
Snap2Code(Unseen)	Layout Accuracy	1.76	3.99
	Visual Fidelity	1.78	4.00
	Content Alignment	2.33	4.03
	Code Usability	1.93	4.11

5.4.2 Survey Result. We received 480 sets of ratings from 4 participants, with each set containing 4 ratings, resulting in a total of 1920 ratings. On average, the participants spent 2 hours completing the questionnaire. The evaluation results are shown in Table 5. We observe that LayoutCoder achieves the highest score in Code Usability on the Design2Code dataset, with a score of 4.01 and an improvement of 14.31% over Claude(SR). It indicates that participants believe the generated code by LayoutCoder is useful for developers on Design2Code (good level). In the Snap2Code(Seen) dataset, LayoutCoder significantly outperforms Claude(SR) with improvements of 152.60% in Visual Fidelity and 122.10% in Content Alignment, with average scores exceeding 4.00 (good level). In contrast, Claude(SR)'s performance is poor, with only 1.64 in Visual Fidelity and 1.86 in Content Alignment, indicating that the four participants are relatively dissatisfied with the results of Claude(SR) on Snap2Code(Seen) (marginal level). The results show that **LayoutCoder is better than Claude(SR) at replicating the layout and style of the reference web pages while maintaining content consistency and code usability.**

6 Discussion

6.1 Why Does Our Model Work?

In this paper, we provide several generation cases to demonstrate the effectiveness of LayoutCoder in the UI2Code task for web pages.

Observation 1: LayoutCoder better accurately captures the layout information from webpage images. As shown in Figure 9, the first image is sampled from real web pages, while the second and third image display the rendering results of the code generated by LayoutCoder and GPT-4o, respectively. By comparing the layout and components arrangement of three images, we observe that LayoutCoder produces better results compared to GPT-4o. This observation suggests that LayoutCoder can capture layout information of the webpage image than GPT-4o.

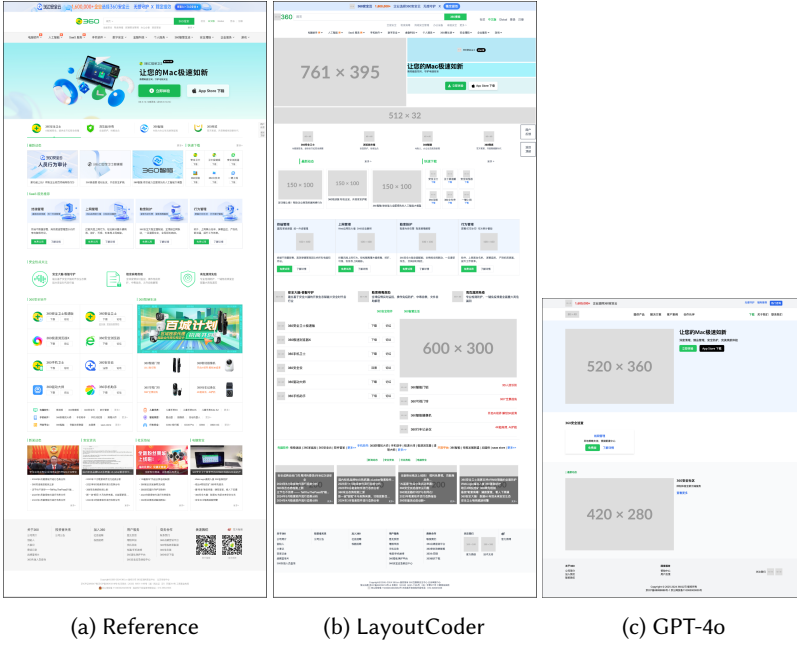


Fig. 9. An example illustrating the difference between the web pages generated by LayoutCoder and GPT-4o.

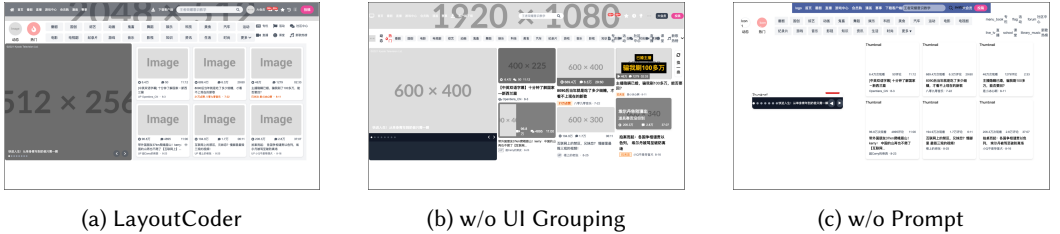


Fig. 10. Examples illustrating the differences between the web pages generated by LayoutCoder and its variants.

Observation 2: LayoutCoder’s UI grouping enhances the visual consistency of repeated components. As shown in Figure 10b, when LayoutCoder excludes UI grouping, similar structural regions are not merged in a UI group. It leads to that six video components with identical structures exhibit visually inconsistent styles. The absence of UI grouping results in an over-segmentation of the webpage image, mistakenly treating the six video components as six independent structures. This indicates that UI grouping enhances the visual consistency of repeated components.

Observation 3: LayoutCoder’s prompt improves the structural compatibility between inner divs and outer divs. As shown in Figure 10c, when the prompt is simplified, layout and style of inner divs conflicts with those of outer divs. This lack of the complete prompt disrupts code generation and visual rendering uniformity. This highlights the LayoutCoder’s complete prompt improves the structural compatibility between inner divs and outer divs.

6.2 Key Differences between LayoutCoder and Other Existing Methods

As shown in Table 6, the three methods, LayoutCoder, DCGen[32], and Screen Parsing[35], are analyzed in terms of three key components: layout extraction, partial code generation, and code fusion.

Table 6. The key differences between LayoutCoder, DCGen and Screen Parsing.

	Layout Extraction	Partial Code Generation	Code Fusion	Scenarios
Screen Parsing	Deep Learning (data-dependent)	/	/	Mobile
DCGen	Separation Line Detection Algorithm	MLLM for code generation	MLLM-based Code Fusion	Web
LayoutCoder	Projection Division Algorithm	MLLM for code generation	Code Fusion Algorithm	Web

Table 7. Comparison of time efficiency, token usage, and generation success rate across two methods.

Method	Dataset	Time	Prompt Tokens	Completion Tokens	Total Tokens	Cost(Dollars)	Generation Success Rate
DCGen	Design2Code	120.12	9752	3870	13622	6.31	89.20%
	Snap2Code(Seen)	1214.72	31457	8989	40446	16.85	60.00%
	Snap2Code(Unseen)	398.59	12314	4139	16453	7.22	91.00%
LayoutCoder	Design2Code	58.31	6989	1156	8145	2.90	100.00%
	Snap2Code(Seen)	327.50	20153	5167	25320	10.21	100.00%
	Snap2Code(Unseen)	155.41	8522	1516	10037	3.65	100.00%

Key difference 1: For Layout Extraction, Screen Parsing relies on deep learning, requiring large, high-quality training data and being UI-specific. In contrast, both DCGen and LayoutCoder are data-independent. However, DCGen struggles with complex UIs due to its reliance on line segmentation thresholds, whereas LayoutCoder uses a projection-based method, effectively handling deep UI structures.

Key difference 2: For Partial Code Generation, Screen Parsing does not generate code but only extracts mobile UI layouts, while both DCGen and LayoutCoder use MLLMs for code generation.

Key difference 3: For Code Fusion, Screen Parsing does not generate code. DCGen uses MLLMs but is limited by token-length constraints for complex webpages, whereas LayoutCoder bypasses MLLM limitations by using decoupling relationships between div elements, enabling more effective code fusion.

The primary contribution of LayoutCoder is its ability to extract layouts without relying on training data, making it highly adaptable to any webpage across both mobile and desktop platforms. Furthermore, LayoutCoder employs a non-MLLM-based code fusion approach, enabling the generation of webpages of arbitrary length, thus overcoming the token length limitations imposed by MLLMs.

6.3 Comparison between LayoutCoder and DCGen

We conducted experiments on the Design2Code, Snap2Code(Seen), and Snap2Code(Unseen) datasets, recording time and token consumption during the generation process. In small-scale trials with DCGen, we observed a higher failure rate and significantly increased time and token consumption on datasets with a high Aspect Ratio. These issues stem from DCGen's reliance on MLLM-based Code Fusion, which is limited by the MLLM's context window length.

Further analysis was conducted using a subset of 50 samples from the Snap2Code(Seen) dataset, which has a higher Aspect Ratio. For datasets with a lower Aspect Ratio, such as Design2Code and Snap2Code(Unseen), full-scale experiments were performed. As shown in Table 7, DCGen's time consumption on different datasets was 2-3 times higher than LayoutCoder's, with increased token consumption and economic costs. DCGen also exhibited a 60% success rate on Snap2Code(Seen), while LayoutCoder achieved 100%.

To ensure fairness, we excluded failed DCGen cases from the sample. Table 8 compares the performance of DCGen and LayoutCoder on two datasets. The results demonstrate that LayoutCoder outperforms DCGen in terms of time efficiency, token consumption, economic cost, code generation success rate, and overall performance across both datasets.

Table 8. Performance comparison between DCGen and LayoutCoder (excluding failures). (%)

Dataset	CLIP		BLEU	
	DCGen	LayoutCoder	DCGen	LayoutCoder
Design2Code(223)	79.28	81.11	1.94	5.08
Snap2Code(Seen)(30)	76.76	80.03	3.98	21.05
Snap2Code(Unseen)(91)	74.34	75.56	1.51	5.70

6.4 Threats to Validity

We identified the following three situations that pose potential threats to the effectiveness of LayoutCoder.

6.4.1 The Selection of Baselines. In this paper, we present a comparison of LayoutCoder, against three baselines. These baselines are all representative and demonstrate superior performance in the existing dataset. Due to our limited computing resources at the time of the paper writing, we do not conduct experiments with a number of other MLLMs such as LLaVA1.5. In the future, we will conduct more experiments on wider types of baselines.

6.4.2 Generalizability on Webpages. The previous works use Design2Code datasets to generate UI code from real-world webpage images. Following the previous methods [24, 32, 47], we limit the size of each dataset to 250 samples to control the experimental budget. To ensure a robust evaluation, we further collect Snap2Code, retains the original images within the web pages. It presents a more realistic and challenging scenario for the UI2Code task, aiming to improve the generalizability of LayoutCoder across various webpages.

6.4.3 Metric Bias. In this paper, we employ BLEU and CLIP scores to assess code similarity and visual consistency in the UI2Code task. While widely used in natural language processing and computer vision, these metrics may not fully capture the nuances of UI2Code. Due to the lack of a standardized evaluation metric for this task, our approach relies on these measures and human assessment, which introduces potential metric bias. In future work, we aim to propose a new evaluation metric specifically for UI2Code, offering a more robust and reliable framework.

7 Conclusion

In this paper, we introduced LayoutCoder, a novel MLLM-based framework generating UI code from real-world webpage image, which includes three key modules: (1) Element Relation Construction, (2) UI Layout Parsing, and (3) Layout-Guided Code Fusion. These modules can improve the ability to understand complex layouts and generate the accurate code with layout preserved. Experimental results demonstrate the superiority of LayoutCoder over the best baseline, Claude 3.5 Sonnet in both visual and code similarity on real-world web pages and robust generalization on highly complex web pages.

Data Availability

The artifacts are archived at this link [34].

Acknowledgments

This research is supported by National Key R&D Program of China (No. 2022YFB3103900), National Natural Science Foundation of China under project (No. 62472126), Natural Science Foundation of Guangdong Province (Project No. 2023A1515011959), Shenzhen-Hong Kong Jointly Funded Project (Category A, No. SGD20230116091246007), Shenzhen Basic Research (General Project No. JCYJ20220531095214031), Shenzhen International Science and Technology Cooperation Project (No. GJHZ20220913143008015), the Major Key Project of PCL (Grant No. PCL2024A05), and CCF-Huawei Populus Grove Fund.

References

- [1] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, et al. 2024. Flamingo: a visual language model for few-shot learning. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '22). Curran Associates Inc., Red Hook, NY, USA, Article 1723, 21 pages.
- [2] Anthropic. 2024. Claude 3.5. <https://www.anthropic.com>. Accessed: 2024-10-30.
- [3] Batuhan Aşıroğlu, Büşta Rümeysa Mete, Eyyüp Yıldız, Yağız Nalçakan, Alper Sezen, Mustafa Dağtekin, and Tolga Ensari. 2019. Automatic HTML code generation from mock-up images using machine learning techniques. In *2019 Scientific Meeting on Electrical-Electronics & Biomedical Engineering and Computer Science (EBBT)*. Ieee, 1–4.
- [4] Microsoft Azure. 2018. Turn your whiteboard sketches to working code in seconds with sketch2code. <https://azure.microsoft.com/en-us/blog/turn-your-whiteboard-sketches-to-working-code-in-seconds-with-sketch2code/> Accessed: 2024-10-30.
- [5] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. 2023. Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond. arXiv:2308.12966 [cs.CV] <https://arxiv.org/abs/2308.12966>
- [6] Daniel Baulé, Christiane Gresse von Wangenheim, Aldo von Wangenheim, Jean C. R. Hauck, and Edson C. Vargas Júnior. 2021. Automatic code generation from sketches of mobile applications in end-user development using Deep Learning. arXiv:2103.05704 [cs.HC] <https://arxiv.org/abs/2103.05704>
- [7] Tony Beltramelli. 2018. pix2code: Generating Code from a Graphical User Interface Screenshot. In *Proceedings of the ACM SIGCHI Symposium on Engineering Interactive Computing Systems* (Paris, France) (EICS '18). Association for Computing Machinery, New York, NY, USA, Article 3, 6 pages. doi:10.1145/3220134.3220135
- [8] Chunyang Chen, Ting Su, Guozhu Meng, Zhenchang Xing, and Yang Liu. 2018. From UI design image to GUI skeleton: a neural machine translator to bootstrap mobile GUI implementation. In *Proceedings of the 40th International Conference on Software Engineering* (Gothenburg, Sweden) (ICSE '18). Association for Computing Machinery, New York, NY, USA, 665–676. doi:10.1145/3180155.3180240
- [9] Wen-Yin Chen, Pavol Podstreleny, Wen-Huang Cheng, Yung-Yao Chen, and Kai-Lung Hua. 2022. Code generation from a graphical user interface via attention-based encoder-decoder model. *Multimedia Systems* 28, 1 (2022), 121–130.
- [10] André Armstrong Janino Cizotto, Rodrigo Clemente Thom de Souza, Viviana Cocco Mariani, and Leandro dos Santos Coelho. 2023. Web pages from mockup design based on convolutional neural network and class activation mapping. 82, 25 (March 2023), 38771–38797. doi:10.1007/s11042-023-15108-3
- [11] Biplab Deka, Zifeng Huang, Chad Franzen, Joshua Hibschan, Daniel Afegan, Yang Li, Jeffrey Nichols, and Ranjitha Kumar. 2017. Rico: A Mobile App Dataset for Building Data-Driven Design Applications. In *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology* (Québec City, QC, Canada) (UIST '17). Association for Computing Machinery, New York, NY, USA, 845–854. doi:10.1145/3126594.3126651
- [12] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. 1996. A density-based algorithm for discovering clusters in large spatial databases with noise (KDD'96). AAAI Press, 226–231.
- [13] Jack Hessel, Ari Holtzman, Maxwell Forbes, Ronan Le Bras, and Yejin Choi. 2022. CLIPScore: A Reference-free Evaluation Metric for Image Captioning. arXiv:2104.08718 [cs.CV] <https://arxiv.org/abs/2104.08718>
- [14] Vanita Jain, Piyush Agrawal, Subham Banga, Rishabh Kapoor, and Shashwat Gulyani. 2019. Sketch2Code: Transformation of Sketches to UI in Real-time Using Deep Neural Network. arXiv:1910.08930 [cs.CV] <https://arxiv.org/abs/1910.08930>
- [15] Zhaoyun Jiang, Jiaqi Guo, Shizhao Sun, Huayu Deng, Zhongkai Wu, Vuksan Mijovic, Zijiang James Yang, Jian-Guang Lou, and Dongmei Zhang. 2023. LayoutFormer++: Conditional Graphic Layout Generation via Constraint Serialization and Decoding Space Restriction. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, Los Alamitos, CA, USA, 18403–18412. doi:10.1109/CVPR52729.2023.01765
- [16] Zhaoyun Jiang, Shizhao Sun, Jihua Zhu, Jian-Guang Lou, and Dongmei Zhang. 2022. Coarse-to-Fine Generative Modeling for Graphic Layouts. *Proceedings of the AAAI Conference on Artificial Intelligence* 36, 1 (Jun. 2022), 1096–1103. doi:10.1609/aaai.v36i1.19994
- [17] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. 2023. Segment Anything. arXiv:2304.02643 [cs.CV] <https://arxiv.org/abs/2304.02643>
- [18] Hugo Laurençon, Léo Tronchon, and Victor Sanh. 2024. Unlocking the conversion of Web Screenshots into HTML Code with the WebSight Dataset. arXiv:2403.09029 [cs.HC] <https://arxiv.org/abs/2403.09029>
- [19] Kenton Lee, Mandar Joshi, Iulia Turc, Hexiang Hu, Fangyu Liu, et al. 2023. Pix2Struct: screenshot parsing as pretraining for visual language understanding. In *Proceedings of the 40th International Conference on Machine Learning* (Honolulu, Hawaii, USA) (ICML '23). JMLR.org, Article 780, 20 pages.
- [20] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. arXiv:2301.12597 [cs.CV] <https://arxiv.org/abs/2301.12597>

- [21] Jianan Li, Jimei Yang, Aaron Hertzmann, Jianming Zhang, and Tingfa Xu. 2019. LayoutGAN: Generating Graphic Layouts with Wireframe Discriminators. arXiv:1901.06767 [cs.CV] <https://arxiv.org/abs/1901.06767>
- [22] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual Instruction Tuning. *ArXiv abs/2304.08485* (2023). <https://api.semanticscholar.org/CorpusID:258179774>
- [23] Kevin Moran, Carlos Bernal-Cárdenas, Michael Curcio, Richard Bonett, and Denys Poshyvanyk. 2018. Machine Learning-Based Prototyping of Graphical User Interfaces for Mobile Apps. *IEEE Transactions on Software Engineering* 46 (2018), 196–221. <https://api.semanticscholar.org/CorpusID:3629347>
- [24] Tuan Anh Nguyen and Christoph Csallner. 2015. Reverse Engineering Mobile Application User Interfaces with REMAUI (T). *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (2015), 248–259. <https://api.semanticscholar.org/CorpusID:7499368>
- [25] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, et al. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] <https://arxiv.org/abs/2303.08774>
- [26] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. BLEU: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics* (Philadelphia, Pennsylvania) (*ACL '02*). Association for Computational Linguistics, USA, 311–318. doi:10.3115/1073083.1073135
- [27] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PMLR, 8748–8763.
- [28] Alex Robinson. 2019. Sketch2code: Generating a website from a paper mockup. arXiv:1905.13750 [cs.CV] <https://arxiv.org/abs/1905.13750>
- [29] Andy Rutledge. 2009. Gestalt Principles - 3: Proximity, Uniform Connectedness, and Good Continuation. <https://andyrutledge.com/gestalt-principles-3.html>. Accessed: 2024-10-30.
- [30] Chenglei Si, Yanzhe Zhang, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. 2024. Design2Code: How Far Are We From Automating Front-End Engineering? *arXiv preprint arXiv:2403.03163* (2024).
- [31] Digital Silk. 2024. How Many Websites Are There In 2024? <https://www.digitalsilk.com/digital-trends/how-many-websites-are-there/>. Accessed: 2024-10-31.
- [32] Yuxuan Wan, Chaozheng Wang, Yi Dong, Wenxuan Wang, Shuqing Li, Yintong Huo, and Michael R. Lyu. 2024. Automatically Generating UI Code from Screenshot: A Divide-and-Conquer-Based Approach. arXiv:2406.16386 [cs.SE] <https://arxiv.org/abs/2406.16386>
- [33] Weihai Wang, Qingsong Lv, Wenmeng Yu, Wenyi Hong, Ji Qi, Yan Wang, Junhui Ji, Zhuoyi Yang, Lei Zhao, Xixuan Song, et al. 2023. Cogvlm: Visual expert for pretrained language models. *arXiv preprint arXiv:2311.03079* (2023).
- [34] Fan Wu. 2025. MLLM-Based UI2Code Automation Guided by UI Layout Information. <https://github.com/ay7u1009/LayoutCoder/>. Accessed: 2025-04-05.
- [35] Jason Wu, Xiaoyi Zhang, Jeff Nichols, and Jeffrey P Bigham. 2021. Screen Parsing: Towards Reverse Engineering of UI Models from Screenshots. In *The 34th Annual ACM Symposium on User Interface Software and Technology* (Virtual Event, USA) (*UIST '21*). Association for Computing Machinery, New York, NY, USA, 470–483. doi:10.1145/3472749.3474763
- [36] Shuhong Xiao, Yunnong Chen, Jiazhi Li, Liuqing Chen, Lingyun Sun, and Tingting Zhou. 2024. Prototype2Code: End-to-end Front-end Code Generation from UI Design Prototypes. arXiv:2405.04975 [cs.SE] <https://arxiv.org/abs/2405.04975>
- [37] Shuhong Xiao, Yunnong Chen, Yaxuan Song, Liuqing Chen, Lingyun Sun, Yankun Zhen, and Yanfang Chang. 2024. UI Semantic Group Detection: Grouping UI Elements with Similar Semantics in Mobile Graphical User Interface. arXiv:2403.04984 [cs.SE] <https://arxiv.org/abs/2403.04984>
- [38] Mulong Xie, Sidong Feng, Zhenchang Xing, Jieshan Chen, and Chunyang Chen. 2020. UIED: a hybrid tool for GUI element detection (*ESEC/FSE 2020*). Association for Computing Machinery, New York, NY, USA, 1655–1659. doi:10.1145/3368089.3417940
- [39] Mulong Xie, Zhenchang Xing, Sidong Feng, Xiwei Xu, Liming Zhu, and Chunyang Chen. 2022. Psychologically-inspired, unsupervised inference of perceptual groups of GUI widgets from GUI images (*ESEC/FSE 2022*). Association for Computing Machinery, New York, NY, USA, 332–343. doi:10.1145/3540250.3549138
- [40] Jianwei Yang, Jiasen Lu, Stefan Lee, Dhruv Batra, and Devi Parikh. 2018. Graph R-CNN for Scene Graph Generation. In *Proceedings of the European Conference on Computer Vision (ECCV)*.
- [41] Hao Zhang, Feng Li, Shilong Liu, Lei Zhang, Hang Su, Jun Zhu, Lionel M. Ni, and Heung-Yeung Shum. 2022. DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection. arXiv:2203.03605 [cs.CV] <https://arxiv.org/abs/2203.03605>
- [42] Junyi Zhang, Jiaqi Guo, Shizhao Sun, Jian-Guang Lou, and D. Zhang. 2023. LayoutDiffusion: Improving Graphic Layout Generation by Discrete Diffusion Probabilistic Models. *2023 IEEE/CVF International Conference on Computer Vision (ICCV)* (2023), 7192–7202. <https://api.semanticscholar.org/CorpusID:257636725>
- [43] Wei Zhang, Shangmin Luan, Liqin Tian, and Nima Jafari Navimipour. 2022. A Rapid Combined Model for Automatic Generating Web UI Codes. *Wirel. Commun. Mob. Comput.* 2022 (Jan. 2022), 10 pages. doi:10.1155/2022/4415479

- [44] Xiaoyi Zhang, Lilian de Greef, Amanda Swearngin, et al. 2021. Screen Recognition: Creating Accessibility Metadata for Mobile Applications from Pixels.. In *CHI*, Yoshifumi Kitamura, Aaron Quigley, Katherine Isbister, Takeo Igarashi, Pernille Bjørn, and Steven Mark Drucker (Eds.). ACM, 275:1–275:15. <http://dblp.uni-trier.de/db/conf/chi/chi2021.html#ZhangGSWMYSNWF21>
- [45] Shuyu Zheng, Ziniu Hu, and Yun Ma. 2019. FaceOff: Assisting the Manifestation Design of Web Graphical User Interface. In *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining* (Melbourne VIC, Australia) (*WSDM '19*). Association for Computing Machinery, New York, NY, USA, 774–777. doi:10.1145/3289600.3290610
- [46] Xu Zhong, Jianbin Tang, and Antonio Jimeno-Yepes. 2019. PubLayNet: Largest Dataset Ever for Document Layout Analysis. *2019 International Conference on Document Analysis and Recognition (ICDAR)* (2019), 1015–1022. <https://api.semanticscholar.org/CorpusID:201124789>
- [47] Ting Zhou, Yanjie Zhao, Xinyi Hou, Xiaoyu Sun, Kai Chen, and Haoyu Wang. 2024. Bridging Design and Development with Automated Declarative UI Code Generation. arXiv:2409.11667 [cs.SE] <https://arxiv.org/abs/2409.11667>

Received 2024-10-31; accepted 2025-03-31